

Министерство науки и высшего образования Российской Федерации
ИТМО

Лабораторная работа №4

C++ PROCESSES / THREADS

по дисциплине:
"C++ и UNIX системы"

Выполнил:
Студент 3 курса, группы Z3300
Заварина Софья

Преподаватель:
Маслов И.Д.

Санкт-Петербург
2026

Цель

Познакомить студента с принципами параллельных вычислений. Составить несколько программ в простейших вычислительных действиях, чтобы освоить принципы параллельных вычислений (когда одни алгоритмы зависят / не зависят от других).

Задача и решение

1. [C++ SEQUENCE] Последовательные вычисления

Требуется последовательно выполнить вычисления по формуле 1, вычисления по формуле 2, после чего выполнить вычисления по формуле 3, которые выглядят следующим образом: результат вычислений 1 + результат вычислений 2 – результат вычислений 1.

Выполнить последовательно на 10 000 итераций и 100 000 итераций.

Формула 1: $f(x) = x^2 - x^2 + x * 4 - x * 5 + x + x$.

Формула 2: $f(x) = x + x$.

Вывести длительность выполнения всех 10 000 итераций и 100 000 итераций в сек.

```
#include <iostream>
#include <chrono>

double formula_1(double x) {
    return x*x - x*x + 4*x - 5*x + x + x;
}

double formula_2(double x) {
    return x + x;
}

double formula_3(double x) {
    return formula_1(x) + formula_2(x) - formula_1(x);
}

int main() {

    double x = 1.234;
    int i;
    volatile double result = 0;

    // 10000 итераций

    auto start_10000 = std::chrono::high_resolution_clock::now();

    for (i = 1; i <= 10000; i++) {
        result = formula_1(x);
    }

    for (i = 1; i <= 10000; i++) {
        result = formula_2(x);
    }

    for (i = 1; i <= 10000; i++) {
        result = formula_3(x);
    }
```

```

    auto end_10000 = std::chrono::high_resolution_clock::now();

    // 100000 итераций

    auto start_100000 = std::chrono::high_resolution_clock::now();

    for (i = 1; i <= 100000; i++) {
        result = formula_1(x);
    }

    for (i = 1; i <= 100000; i++) {
        result = formula_2(x);
    }

    for (i = 1; i <= 100000; i++) {
        result = formula_3(x);
    }

    auto end_100000 = std::chrono::high_resolution_clock::now();

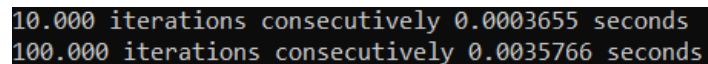
    std::chrono::duration<double> iterations_10000 = end_10000 - start_10000;
    std::chrono::duration<double> iterations_100000 = end_100000 - start_100000;

    // results

    std::cout << "10.000 iterations consecutively " << iterations_10000.count() << " seconds" << std::endl;
    std::cout << "100.000 iterations consecutively " << iterations_100000.count() << " seconds" << std::endl;

    return 0;
}

```



```

10.000 iterations consecutively 0.0003655 seconds
100.000 iterations consecutively 0.0035766 seconds

```

Рис. 1: Вывод

2. [C++ THREADS] Параллельные вычисления через потоки

Требуется параллельно (насколько возможно с помощью потоков) выполнить вычисления по формуле 1, вычисления по формуле 2, после чего выполнить вычисления по формуле 3, которые выглядят следующим образом: результат вычислений 1 + результат вычислений 2 – результат вычислений 1.

Выполнить последовательно на 10 000 итераций и 100 000 итераций.

Формула 1: $f(x) = x^2 - x^2 + x * 4 - x * 5 + x + x$

Формула 2: $f(x) = x + x$

Вывести длительность выполнения всех 10 000 итераций и 100 000 итераций в сек. в разбивке по шагам вычислений 1, 2 и 3.

```

#include <iostream>
#include <chrono>
#include <thread>

double formula_1(double x) {
    return x*x - x*x + 4*x - 5*x + x + x;
}

double formula_2(double x) {

```

```

    return x + x;
}

double formula_3(double x) {
    return formula_1(x) + formula_2(x) - formula_1(x);
}

int main() {

    double x = 1.234;
    volatile double result = 0;

    //10 000 итераций

    auto start_10000 = std::chrono::high_resolution_clock::now();

    std::thread t1([&]() {
        for (int i = 1; i <= 10000; i++) {
            result = formula_1(x);
        }
    });

    std::thread t2([&]() {
        for (int i = 1; i <= 10000; i++) {
            result = formula_2(x);
        }
    });

    std::thread t3([&]() {
        for (int i = 1; i <= 10000; i++) {
            result = formula_3(x);
        }
    });

    t1.join();
    t2.join();
    t3.join();

    auto end_10000 = std::chrono::high_resolution_clock::now();

    //100 000 итераций

    auto start_100000 = std::chrono::high_resolution_clock::now();

    std::thread t4([&]() {
        for (int i = 1; i <= 100000; i++) {
            result = formula_1(x);
        }
    });

    std::thread t5([&]() {
        for (int i = 1; i <= 100000; i++) {
            result = formula_2(x);
        }
    });
}

```

```

std::thread t6([&]() {
    for (int i = 1; i <= 100000; i++) {
        result = formula_3(x);
    }
});

t4.join();
t5.join();
t6.join();

auto end_100000 = std::chrono::high_resolution_clock::now();

std::chrono::duration<double> iterations_10000 = end_10000 - start_10000;
std::chrono::duration<double> iterations_100000 = end_100000 - start_100000;

std::cout << "10.000 iterations parallel " << iterations_10000.count() << " seconds" << std::endl;
std::cout << "100.000 iterations parallel " << iterations_100000.count() << " seconds" << std::endl;

return 0;
}

```

```

10.000 iterations parallel 0.0012142 seconds
100.000 iterations parallel 0.0066167 seconds

```

Рис. 2: Вывод

3. [C++ PROCESS] Параллельные вычисления через процессы

Требуется параллельно (насколько возможно с помощью процессов) выполнить вычисления по формуле 1, вычисления по формуле 2, после чего выполнить вычисления по формуле 3, которые выглядят следующим образом: результат вычислений 1 + результат вычислений 2 – результат вычислений 1.

Выполнить последовательно на 10 000 итераций и 100 000 итераций.

Формула 1: $f(x) = x^2 - x^2 + x * 4 - x * 5 + x + x$

Формула 2: $f(x) = x + x$

Вывести длительность выполнения всех 10 000 итераций и 100 000 итераций в сек. в разбивке по шагам вычислений 1, 2 и 3.

```

#include <iostream>
#include <chrono>
#include <windows.h>

double formula_1(double x) {
    return x*x - x*x + 4*x - 5*x + x + x;
}

double formula_2(double x) {
    return x + x;
}

int main() {
    double x = 1.234;

    //10 000 итераций =====

    auto start_10000 = std::chrono::high_resolution_clock::now();

```

```

HANDLE file_10000 = CreateFileMappingA(
    INVALID_HANDLE_VALUE,
    NULL,
    PAGE_READWRITE,
    0,
    sizeof(double) * 10000 * 2,
    "MemoryFor10000"
);

double* shared = (double*)MapViewOfFile(
    file_10000,
    FILE_MAP_ALL_ACCESS,
    0, 0,
    sizeof(double) * 10000 * 2
);

STARTUPINFOA si1 = {sizeof(si1)};
PROCESS_INFORMATION pi1;
CreateProcessA(
    NULL,
    (LPSTR)"child.exe 1 10000",
    NULL, NULL, FALSE, 0, NULL, NULL, &si1, &pi1
);

STARTUPINFOA si2 = {sizeof(si2)};
PROCESS_INFORMATION pi2;
CreateProcessA(
    NULL,
    (LPSTR)"child.exe 2 10000",
    NULL, NULL, FALSE, 0, NULL, NULL, &si2, &pi2
);

WaitForSingleObject(pi1.hProcess, INFINITE);
WaitForSingleObject(pi2.hProcess, INFINITE);

volatile double r3_10000 = 0;
for (int i = 0; i < 10000; i++) {
    double r1 = shared[i];
    double r2 = shared[i + 10000];
    r3_10000 = r1 + r2 - r1;
}

auto end_10000 = std::chrono::high_resolution_clock::now();

CloseHandle(pi1.hProcess);
CloseHandle(pi1.hThread);
CloseHandle(pi2.hProcess);
CloseHandle(pi2.hThread);
UnmapViewOfFile(shared);
CloseHandle(file_10000);

//100 000 итераций

auto start_100000 = std::chrono::high_resolution_clock::now();

```

```

HANDLE file_100000 = CreateFileMappingA(
    INVALID_HANDLE_VALUE,
    NULL,
    PAGE_READWRITE,
    0,
    sizeof(double) * 100000 * 2,
    "MemoryFor100000"
);

double* shared2 = (double*)MapViewOfFile(
    file_100000,
    FILE_MAP_ALL_ACCESS,
    0, 0,
    sizeof(double) * 100000 * 2
);

STARTUPINFOA si3 = {sizeof(si3)};
PROCESS_INFORMATION pi3;
CreateProcessA(
    NULL,
    (LPSTR)"child.exe 1 100000",
    NULL, NULL, FALSE, 0, NULL, NULL, &si3, &pi3
);

STARTUPINFOA si4 = {sizeof(si4)};
PROCESS_INFORMATION pi4;
CreateProcessA(
    NULL,
    (LPSTR)"child.exe 2 100000",
    NULL, NULL, FALSE, 0, NULL, NULL, &si4, &pi4
);

WaitForSingleObject(pi3.hProcess, INFINITE);
WaitForSingleObject(pi4.hProcess, INFINITE);

volatile double r3_100000 = 0;
for (int i = 0; i < 100000; i++) {
    double r1 = shared2[i];
    double r2 = shared2[i + 100000];
    r3_100000 = r1 + r2 - r1;
}

auto end_100000 = std::chrono::high_resolution_clock::now();

CloseHandle(pi3.hProcess);
CloseHandle(pi3.hThread);
CloseHandle(pi4.hProcess);
CloseHandle(pi4.hThread);
UnmapViewOfFile(shared2);
CloseHandle(file_100000);

std::chrono::duration<double> time_10000 = end_10000 - start_10000;
std::chrono::duration<double> time_100000 = end_100000 - start_100000;

std::cout << "10.000 iterations processes " << time_10000.count() << " seconds" << std::endl;
std::cout << "100.000 iterations processes " << time_100000.count() << " seconds" << std::endl;

```

```

    return 0;
}

#include <iostream>
#include <windows.h>
#include <cstdlib>

double formula_1(double x) {
    return x*x - x*x + 4*x - 5*x + x + x;
}

double formula_2(double x) {
    return x + x;
}

int main(int argc, char* argv[]) {
    if (argc != 3) {
        std::cerr << "Usage: child.exe <formula> <iterations>" << std::endl;
        return 1;
    }

    int formula = atoi(argv[1]);
    int n = atoi(argv[2]);

    HANDLE hMap;
    if (n == 10000) {
        hMap = OpenFileMappingA(FILE_MAP_ALL_ACCESS, FALSE, "MemoryFor10000");
    } else {
        hMap = OpenFileMappingA(FILE_MAP_ALL_ACCESS, FALSE, "MemoryFor100000");
    }

    if (hMap == NULL) {
        std::cerr << "Не удалось открыть память" << std::endl;
        return 1;
    }

    double* shared = (double*)MapViewOfFile(
        hMap, FILE_MAP_ALL_ACCESS, 0, 0,
        sizeof(double) * n * 2
    );

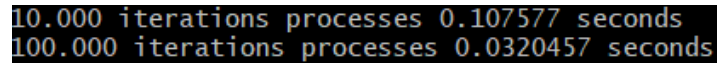
    double x = 1.234;

    if (formula == 1) {
        for (int i = 0; i < n; i++) {
            shared[i] = formula_1(x);
        }
    } else {
        for (int i = 0; i < n; i++) {
            shared[i + n] = formula_2(x);
        }
    }

    UnmapViewOfFile(shared);
    CloseHandle(hMap);
    return 0;
}

```

}



```
10.000 iterations processes 0.107577 seconds
100.000 iterations processes 0.0320457 seconds
```

Рис. 3: Вывод

[LOG] Результат всех вышеперечисленных шагов сохранить в репозиторий (+ отчет по данной ЛР в папку doc)

Заключение

В ходе лабораторной работы я разобралась с тремя подходами к вычислениям — последовательным, параллельным через потоки и параллельным через процессы.

Последовательные вычисления. Программа для 10 000 и 100 000 итераций по очереди считает формулу 1, потом формулу 2, потом формулу 3. Время замеряется через `std::chrono`. Простой вариант — но не использует преимущества многоядерных процессоров.

Параллельные через потоки (`std::thread`). Формулы 1 и 2 считаются одновременно, в разных потоках внутри одной программы. У потоков общая память — обмениваться данными просто. Главный поток ждёт завершения через `join()`. Это работает и в Linux, и в Windows.

Параллельные через процессы (`CreateProcess`). Здесь две отдельные программы — `main.exe` и `child.exe`. Они изолированы, поэтому общаются через разделяемую память (`CreateFileMapping` + `MapViewOfFile`). Главная программа запускает два дочерних процесса (`child.exe` 1 и `child.exe` 2), ждёт их через `WaitForSingleObject`, потом читает результаты из памяти и считает формулу 3. Это работает только в Windows — нужен `windows.h`.

В целом работа дала понимание, как устроена параллельность на уровне ОС — и чем отличаются потоки от процессов.